

LAPORAN PRAKTIKUM ALGORITMA DAN PEMROGRAMAN

PROJEK AKHIR TUTOR: SISTEM ABSENSI DAN REKAPITULASI SISWA BERBASIS C++

Disusun oleh:
Kelompok: 8

Anggota Kelompok:

- Amirah - 140810260002
- Azka - 140810260085
- izza - 140810260003
- Hana - 140810260061
- John - 140810260083
- nael - 140810260018
- Falaah - 140810260075
- Yudish - 140810260091
- Uen - 140810260009

Tutor: Kang Rakhi

BAB I: PENDAHULUAN

1.1 Latar Belakang

Pengelolaan data kehadiran siswa secara konsisten merupakan bagian penting dalam administrasi pembelajaran. Pencatatan manual menggunakan kertas sering kali menimbulkan kendala seperti hilangnya riwayat absensi harian, kesalahan dalam rekapitulasi data, serta sulitnya mengurutkan data berdasarkan nomor induk siswa.

Seiring dengan pemahaman materi dasar pemrograman C++, permasalahan tersebut dapat diatasi dengan membangun **Sistem Absensi dan Rekapitulasi Siswa Berbasis Konsol**. Aplikasi ini memanfaatkan konsep pemodelan data terstruktur menggunakan *Array* serta pengelolaan fungsi modular guna mempermudah pencatatan kehadiran harian, perhitungan rekapitulasi, dan pengurutan data berdasarkan NPM.

1.2 Rumusan Masalah

1. Bagaimana merancang program absensi harian dan rekapitulasi siswa yang efisien menggunakan bahasa C++?
2. Bagaimana mengimplementasikan metode *Computational Thinking* dalam menyelesaikan permasalahan pengelolaan data absensi?

3. Bagaimana mengintegrasikan materi dari 5 pertemuan tutor CBS (seperti I/O, percabangan, perulangan, fungsi, array 1D/2D, serta *sorting*) ke dalam satu program yang utuh?

1.3 Tujuan

1. Memenuhi tugas Projek Akhir Tutor Algoritma dan Pemrograman.
2. Menerapkan 4 pilar *Computational Thinking* dalam merancang solusi perangkat lunak.
3. Menerapkan materi C++ yang dipelajari selama pertemuan tutor, khususnya penggunaan array 1D/2D, *function*, dan algoritma *Bubble Sort*.

BAB II: PENERAPAN COMPUTATIONAL THINKING

Pembangunan aplikasi ini menerapkan 4 pilar *Computational Thinking*:

1. **Decomposition (Dekomposisi)**
Memecah kompleksitas aplikasi menjadi modul-modul fungsi independen:
 - o Modul Menu Utama (tampilanMenu)
 - o Modul Tambah Siswa (tambahSiswa)
 - o Modul Tampil Data Siswa (tampilkanDataSiswa)
 - o Modul Catat Absensi (catatAbsensi)
 - o Modul Rekap Absensi (rekapAbsensi)
 - o Modul Pengurutan Data (urutkanNPM)
2. **Pattern Recognition (Pengenalan Pola)**
 - o Penyimpanan data siswa (NPM dan Nama) memiliki hubungan 1-ke-1 sehingga dapat dipetakan menggunakan *Array 1D* sejajar (parallel arrays).
 - o Pencatatan status kehadiran dilakukan berulang per hari per siswa, yang cocok direpresentasikan menggunakan *Array 2D* (Siswa x Hari).
 - o Pengurutan NPM mengikuti pola pertukaran posisi bersyarat (*Bubble Sort*).
3. **Abstraction (Abstraksi)**
Fokus hanya pada informasi utama yang dibutuhkan sistem (NPM, Nama, dan Kode Status Absensi: 1=Hadir, 2=Izin, 3=Sakit, 4=Alpa). Informasi non-esensial (seperti alamat, umur, atau nilai) tidak dimasukkan dalam struktur data.
4. **Algorithm Design (Perancangan Algoritma)**
Menyusun alur eksekusi berbasis *menu-driven loop* menggunakan perulangan do-while dan percabangan switch-case.

BAB III: PENERAPAN MATERI TUTOR CBS

Program ini mengintegrasikan seluruh materi yang dipelajari dari Pertemuan 1 hingga Pertemuan 5:

1. **Pertemuan 1 & 2: Dasar C++, Variabel, Input/Output, & Operator**
Menggunakan tipe data int untuk NPM dan status absensi, serta string untuk nama siswa. Menggunakan cin, getline(), dan cin.ignore() untuk penanganan input. Menggunakan const int untuk pembatasan kapasitas maksimum (MAX_SISWA = 100, MAX_HARI = 30).
2. **Pertemuan 2: Percabangan (Conditional Statement)**
Menggunakan switch-case pada navigasi menu utama serta pada modul rekapitulasi kehadiran. Menggunakan percabangan if-else untuk validasi kapasitas siswa, batas hari,

dan input pilihan.

3. Pertemuan 3: Perulangan (Looping)

Perulangan do-while untuk menjaga menu utama tetap aktif hingga pengguna memilih keluar (opsi 0) serta untuk validasi input kehadiran. Perulangan for untuk iterasi data array pada modul tampil data, absensi, rekap, dan sorting.

4. Pertemuan 4: Fungsi (Function) & Clean Code

Pemisahan kode ke dalam modul fungsi non-returning (void) dengan parameter *pass-by-value* dan *pass-by-reference* (&jumlahSiswa). Penerapan prinsip *Clean Code*: nama variabel deskriptif, fungsi fokus pada satu tugas (DRY - *Don't Repeat Yourself*), dan penulisan indentasi rapi.

5. Pertemuan 5: Computational Thinking, Array 1D/2D, & Sorting

Array 1D: npm[MAX_SISWA] dan nama[MAX_SISWA] untuk menyimpan data identitas siswa.

Array 2D: absensi[MAX_SISWA][MAX_HARI] untuk menyimpan matriks data absensi harian.

Sorting: Algoritma *Bubble Sort* pada fungsi urutkanNPM untuk mengurutkan NPM secara *ascending*, sekaligus menyinkronkan data nama dan matriks absensi harian agar tidak tertukar.

BAB IV: PERANCANGAN DAN ALUR PROGRAM

4.1 Deskripsi Program

Program **Sistem Absensi Siswa** memiliki 6 menu utama:

- **Tambah Data Siswa:** Memasukkan NPM dan Nama siswa baru.
- **Tampilkan Data Siswa:** Menampilkan daftar siswa yang telah terdaftar.
- **Absensi:** Mencatatkan status kehadiran harian (1=Hadir, 2=Izin, 3=Sakit, 4=Alpa) untuk seluruh siswa.
- **Rekap Absensi:** Menghitung dan menampilkan total frekuensi Hadir, Izin, Sakit, dan Alpa tiap siswa.
- **Urutkan Data Siswa Berdasarkan NPM:** Mengurutkan seluruh record data berdasarkan urutan NPM menggunakan Bubble Sort.
- **Keluar:** Mengakhiri jalannya aplikasi.

4.2 Alur Logika Program (Flowchart / Algoritma)

1. Inisialisasi variabel jumlahSiswa = 0, hari = 0, array npm, nama, dan array 2D absensi.
2. Tampilkan daftar menu utama (tampilanMenu).
3. Pengguna memasukkan pilihan (0–5):
 - Opsi 1: Panggil tambahSiswa(). Cek jumlahSiswa < MAX_SISWA, baca NPM dan Nama, naikan jumlahSiswa.
 - Opsi 2: Panggil tampilkanDataSiswa(). Cetak seluruh isi array npm dan nama.
 - Opsi 3: Panggil catatAbsensi(). Iterasi seluruh siswa, lakukan perulangan validasi input status (1–4), simpan ke absensi[i][hari], lalu naikan variabel hari.
 - Opsi 4: Panggil rekapAbsensi(). Hitung kemunculan angka 1–4 per siswa pada matriks absensi dari hari ke-0 hingga hari - 1, tampilkan dalam format rekapitulasi.
 - Opsi 5: Panggil urutkanNPM(). Jalankan *nested loop* Bubble Sort untuk menukar

- posisi elemen pada array npm, nama, dan seluruh baris pada absensi.
- Opsi 0: Keluar dari perulangan do-while.

BAB V: IMPLEMENTASI KODE PROGRAM (C++)

Berikut adalah kode program C++ lengkap yang diimplementasikan dalam proyek ini:

```
#include <iostream>
#include <string>
using namespace std;

int const MAX_SISWA = 100;
int const MAX_HARI = 30;

// Deklarasi fungsi
void tampilanMenu();
void tambahSiswa(int npm[], string nama[], int &jumlahSiswa);
void tampilkanDataSiswa(int npm[], string nama[], int jumlahSiswa);
void catatAbsensi(string nama[], int absensi[][MAX_HARI], int jumlahSiswa, int hari);
void rekapAbsensi(int npm[], string nama[], int absensi[][MAX_HARI], int jumlahSiswa, int hari);
void urutkanNPM(int npm[], string nama[], int absensi[][MAX_HARI], int jumlahSiswa, int hari);

void tampilanMenu() {
    cout << "===== failure=====\\n";
    cout << "0. Keluar\\n";
    cout << "1. Tambah Data Siswa\\n";
    cout << "2. Tampilkan Data Siswa\\n";
    cout << "3. Absensi\\n";
    cout << "4. Rekap Absensi\\n";
    cout << "5. Urutkan Data Siswa Berdasarkan NPM\\n";
    cout << "=====\\n";
    cout << "Masukan Pilihan Menu (0-5) : ";
}

void tambahSiswa(int npm[], string nama[], int &jumlahSiswa) {
    if (jumlahSiswa >= MAX_SISWA) {
        cout << "Kapasitas siswa penuh!\\n";
        return;
    }

    cout << "Masukkan npm siswa: ";
    cin >> npm[jumlahSiswa];

    cin.ignore();
    cout << "Masukkan nama siswa: ";
    getline(cin, nama[jumlahSiswa]);
```

```

    jumlahSiswa++;
    cout << "Data siswa berhasil ditambahkan!\n";
}

```

```

void tampilkanDataSiswa(int npm[], string nama[], int jumlahSiswa) {
    if (jumlahSiswa == 0) {
        cout << "Belum ada data siswa!\n";
        return;
    }
    for (int i = 0; i < jumlahSiswa; i++) {
        cout << "NPM: " << npm[i] << ", \tNama: " << nama[i] << endl;
    }
}

```

```

void catatAbsensi(string nama[], int absensi[][MAX_HARI], int jumlahSiswa, int hari) {
    cout << "Masukkan data Absensi" << endl;

```

```

    if (jumlahSiswa == 0) {
        cout << "Belum ada data Siswa" << endl;
        return;
    }

```

```

    if (hari >= MAX_HARI) {
        cout << "Batas maksimum pencatatan hari (" << MAX_HARI << " hari) telah tercapai!\n";
        return;
    }

```

```

    cout << "Absensi hari ke-" << hari + 1 << endl;
    cout << "1 = Hadir\n";
    cout << "2 = Izin\n";
    cout << "3 = Sakit\n";
    cout << "4 = Alpa\n\n";

```

```

    for (int i = 0; i < jumlahSiswa; i++) {
        int statusAbsensi;

```

```

        // Validasi input status absensi
        do {
            cout << "Masukkan status absensi untuk " << nama[i] << ": ";
            cin >> statusAbsensi;

```

```

            if (statusAbsensi < 1 || statusAbsensi > 4) {
                cout << "Input tidak valid! Silakan masukkan angka antara 1 hingga 4.\n";
            }
        } while (statusAbsensi < 1 || statusAbsensi > 4);

```

```

        // Simpan data setelah input dipastikan benar

```

```

        absensi[i][hari] = statusAbsensi;
    }
    cout << "Absensi hari ke-" << (hari + 1) << " berhasil disimpan!\n";
}

```

// Fungsi untuk menampilkan rekap absensi

```

void rekapAbsensi(int npm[], string nama[], int absensi[][MAX_HARI], int jumlahSiswa, int hari) {
    if (jumlahSiswa == 0) {
        cout << "Belum ada data siswa!\n";
        return;
    }

```

```

    if (hari == 0) {
        cout << "Belum ada data absensi yang dicatat!\n";
        return;
    }

```

```

    cout << "Rekap Absensi Siswa:\n";
    cout << "NPM\tNama\t\tHadir\tIzin\tSakit\tAlpa\n";

```

```

    for (int i = 0; i < jumlahSiswa; i++) {
        int hadir = 0, izin = 0, sakit = 0, alpa = 0;

```

```

        for (int j = 0; j < hari; j++) {
            switch (absensi[i][j]) {
                case 1: hadir++; break;
                case 2: izin++; break;
                case 3: sakit++; break;
                case 4: alpa++; break;
            }
        }
    }

```

```

        cout << npm[i] << "\t" << nama[i] << "\t\t" << hadir << "\t" << izin << "\t" << sakit << "\t" <<
        alpa << endl;
    }
}

```

// Mengurutkan data absensi berdasarkan npm

```

void urutkanNPM(int npm[], string nama[], int absensi[][MAX_HARI], int jumlahSiswa, int hari)
{
    if (jumlahSiswa == 0) {
        cout << "Belum ada data siswa untuk diurutkan!\n";
        return;
    }

```

```

    if (jumlahSiswa == 1) {
        cout << "Hanya ada satu siswa, tidak perlu diurutkan.\n";
    }

```

```
return;  
}
```

```
// Algoritma Bubble Sort  
for (int i = 0; i < jumlahSiswa - 1; i++) {  
    for (int j = 0; j < jumlahSiswa - i - 1; j++) {  
        if (npm[j] > npm[j + 1]) {  
            // 1. Tukar posisi NPM  
            int tempNPM = npm[j];  
            npm[j] = npm[j + 1];  
            npm[j + 1] = tempNPM;
```

```
            // 2. Tukar posisi Nama (agar tetap sinkron dengan NPM)  
            string tempNama = nama[j];  
            nama[j] = nama[j + 1];  
            nama[j + 1] = tempNama;
```

```
            // 3. Tukar posisi riwayat absensi harian (Array 2D)  
            for (int h = 0; h < hari; h++) {  
                int tempAbsen = absensi[j][h];  
                absensi[j][h] = absensi[j + 1][h];  
                absensi[j + 1][h] = tempAbsen;  
            }  
        }  
    }  
}
```

```
cout << "Data siswa berhasil diurutkan berdasarkan NPM!\n";  
}
```

```
int main() {  
    int jumlahSiswa = 0;  
    int npm[MAX_SISWA];  
    string nama[MAX_SISWA];  
    int hari = 0;  
    int absensi[MAX_SISWA][MAX_HARI];
```

```
    int pilihan;  
    do {  
        tampilanMenu();  
        cin >> pilihan;
```

```
        switch (pilihan) {  
            case 0:  
                cout << "Keluar dari program. Terima kasih!\n";  
                break;  
            case 1:  
                tambahSiswa(npm, nama, jumlahSiswa);
```

```

        break;
    case 2:
        tampilkanDataSiswa(npm, nama, jumlahSiswa);
        break;
    case 3:
        if (jumlahSiswa == 0) {
            cout << "Belum ada data Siswa!\n";
        } else if (hari < MAX_HARI) {
            catatAbsensi(nama, absensi, jumlahSiswa, hari);
            hari++;
        } else {
            cout << "Batas maksimum pencatatan hari telah tercapai!\n";
        }
        break;
    case 4:
        rekapAbsensi(npm, nama, absensi, jumlahSiswa, hari);
        break;
    case 5:
        urutkanNPM(npm, nama, absensi, jumlahSiswa, hari);
        break;
    default:
        cout << "Pilihan tidak valid!\n";
    }
} while (pilihan != 0);

return 0;
}

```

BAB VI: HASIL DAN PEMBAHASAN

6.1 Pengujian Program

Pengujian fungsionalitas dilakukan pada setiap menu program dengan hasil sebagai berikut:

No	Fitur Pengujian	Skenario Uji	Hasil yang Diharapkan	Status
1	Tambah Siswa	Memasukkan NPM (misal: 14081023002) dan Nama	Data tersimpan, jumlahSiswa bertambah	Berhasil
2	Tampilkan Siswa	Memilih menu 2 saat data terisi / kosong	Menampilkan daftar siswa / pesan belum ada data	Berhasil

No	Fitur Pengujian	Skenario Uji	Hasil yang Diharapkan	Status
3	Catat Absensi	Input nilai status 1–4 untuk tiap siswa	Data masuk ke array 2D <code>absensi[i][hari]</code>	Berhasil
4	Validasi Absensi	Input nilai di luar range (misal: 5 atau 0)	Sistem menolak & meminta input ulang (1–4)	Berhasil
5	Rekap Absensi	Memilih menu 4 setelah absensi beberapa hari	Menghitung total Hadir, Izin, Sakit, Alfa per siswa	Berhasil
6	Urutkan NPM	Memilih menu 5 dengan kondisi NPM acak	Data NPM, Nama, dan Absensi terurut secara <i>ascending</i>	Berhasil

BAB VII: PENUTUP

7.1 Kesimpulan

Sistem Absensi dan Rekapitulasi Siswa Berbasis C++ telah berhasil direalisasikan sesuai dengan kriteria Projek Akhir Tutor. Penerapan metode *Computational Thinking* membantu proses pengembangan menjadi lebih sistematis dan terstruktur. Seluruh materi dasar pemrograman C++ (mencakup variabel, I/O, percabangan, perulangan, fungsi modular, array 1D/2D, serta algoritma pengurutan *Bubble Sort*) telah berhasil diimplementasikan secara optimal.

7.2 Saran

1. Menambahkan mekanisme *File Handling* (ifstream/ofstream) agar data siswa dan riwayat absensi dapat tersimpan secara permanen pada direktori lokal.
2. Mengembangkan antarmuka berbasis GUI (*Graphical User Interface*) atau *web-based interface* agar penggunaan aplikasi lebih intuitif.